

CivicReporter: An Integrated, AI-Powered Platform for Crowdsourced Public Infrastructure Fault Detection and Management Using YOLOv11

Navneet Parashar Vindhya Agarwal Manan Tageja
VIT Bhopal University VIT Bhopal University VIT Bhopal University

Tanmay P Khandelwal Sehaj Jain
VIT Bhopal University VIT Bhopal University

Abstract

The maintenance of urban infrastructure is a critical component of metropolitan development, yet traditional methods for identifying and resolving faults are often inefficient and opaque. This paper introduces CivicReporter, a comprehensive full-stack web application designed to streamline the reporting and management of public infrastructure issues by integrating crowdsourced citizen reports with advanced Artificial Intelligence (AI). The system bridges the communication gap between citizens and municipal authorities by implementing a novel dual-engine AI system leveraging You Only Look Once version 11 (YOLOv11) models for automated detection of potholes and garbage from citizen-submitted images.

The methodology employs a sophisticated four-tier, role-based verification workflow built on a FastAPI backend with a vanilla JavaScript frontend. The implemented YOLOv11 models achieve superior performance, with the pothole detection model demonstrating 91% precision and 89% recall, while the garbage detection model achieves 87% precision and 85% recall. Statistical analysis confirms the significance of performance improvements over existing approaches ($p < 0.05$). The findings demonstrate a functional and efficient pipeline where geo-tagged citizen reports are automatically classified, routed for on-ground verification, and managed through a transparent, multi-level ac-

countability framework. CivicReporter provides a scalable and effective model for smart city governance, enhancing transparency, optimizing resource allocation, and fostering a collaborative approach to urban maintenance.

Keywords: infrastructure fault detection; crowdsourcing; computer vision; YOLOv11; smart cities; civic technology; real-time object detection.

1 Introduction

1.1 The Urban Infrastructure Challenge

Urban infrastructure forms the backbone of modern society, yet it is under constant strain from increasing industrialization, urbanization, climate change, and transportation overload [1]. Faults in these critical systems—ranging from minor road cracks to major structural failures—pose direct threats to public safety, disrupt economic activities, and degrade the quality of life [1]. The timely and accurate detection of these faults is paramount for enabling preemptive maintenance, mitigating catastrophic failures, and ensuring the resilience of urban environments [2].

1.2 Limitations of Conventional Reporting Systems

Historically, the identification of infrastructure faults has relied on manual inspections and citizen complaints lodged through conventional channels [1]. These traditional mechanisms are fraught with inefficiencies, characterized by slow response times, opaque processes, and heavy dependence on manual logging. This reactive approach frequently leads to prolonged repair times, unequal service distribution, and growing public frustration [1].

1.3 The Rise of AI and Crowdsourcing in Smart Cities

The convergence of Artificial Intelligence (AI) and widespread citizen connectivity has introduced a paradigm shift in urban management. AI, particularly advancements in deep learning and computer vision, has emerged as a game-changing tool, providing automated, scalable, and real-time solutions for defect detection [1]. Simultaneously, crowdsourcing has proven to be a powerful and cost-effective strategy for large-scale data collection, transforming citizens into active participants in monitoring their communities [1].

1.4 Proposed Solution: The CivicReporter Platform

This paper presents CivicReporter, a novel, integrated platform designed to address the limitations of conventional systems by synergizing citizen reporting, AI-powered analysis, and structured municipal workflow management. The system is built on three foundational pillars: (1) an accessible, web-based portal for citizens to submit geo-tagged photographic evidence; (2) an automated AI engine utilizing state-of-the-art YOLOv11 models; and (3) a unique, multi-tier human-in-the-loop workflow for verification, management, and resolution [1].

1.5 Innovation and Technical Contributions

The primary technical innovation of this work lies in the implementation and optimization of YOLOv11 for infrastructure fault detection. Key contributions include:

(1) a comprehensive performance analysis of YOLOv11 for pothole and garbage detection; (2) a comparative evaluation against established models like YOLOv8, YOLOv5, and Faster R-CNN; (3) statistical significance analysis of performance improvements; and (4) the integration of these advanced AI capabilities within a complete end-to-end municipal workflow system.

2 Background and Literature Review

2.1 Historical Evolution of Infrastructure Monitoring

Infrastructure monitoring has seen a dramatic evolution, beginning with manual, labor-intensive inspections and record-keeping in the mid-20th century [1]. Early digital systems in the late 1980s and 1990s adopted basic logging and polling protocols such as SNMP (Simple Network Management Protocol), enabling scheduled device status checks and threshold-based alerts, but remained largely reactive and siloed [2]. The advent of client-server and web-based computing introduced centralized management platforms, including commercial “Big 4” tools (HP OpenView, IBM Tivoli), as well as open-source solutions like Nagios and Cacti [3]. However, these systems maintained a primary focus on network and hardware uptime rather than direct fault localization and citizen-centric infrastructure issues.

The early 2000s marked a shift towards more proactive, application-focused performance monitoring (APM), with tools such as Dynatrace and AppDynamics enabling deeper analytics and transaction tracing. Yet, these advances were hardly accessible to municipal agencies or non-enterprise domains due to deployment complexity and costs. For physical-world assets like roads and public facilities, traditional and sensor-driven inspections continued to dominate, limited by human bottlenecks, venue coverage constraints, and high operational overheads.

2.2 Sensor-based and AI-driven Defect Detection

The integration of sensors, such as vibration recorders, ultrasonic scanners, and IoT devices, allowed continu-

ous monitoring of the real-world infrastructure. Such systems, widely deployed in bridges and railway networks, provided higher temporal resolution but faced clear constraints: high cost-per-installed-node, complex calibration, limited scalability beyond specialized domains, and the inability to recognize diverse or rare fault types outside preprogrammed anomalies [4].

The explosion of computer vision and, specifically, the triumph of deep learning delivered a pivotal breakthrough. Two-stage detectors such as Faster R-CNN led the early automated defect detection field, achieving high precision but suffering from computational inefficiency—impractical for real-time or large-scale civil infrastructure workflows. The growth of open annotated datasets for road and building environments further accelerated the adoption of AI in automated monitoring, enabling reproducible benchmarks and model development efforts beyond the limits of handcrafted features.

2.3 YOLO Family and Model Progression

The introduction of single-stage object detectors, particularly the YOLO (“You Only Look Once”) family, revolutionized computer vision in applied infrastructure monitoring. YOLOv1 established the paradigm by combining speed and reasonable accuracy, enabling near real-time inference on consumer-grade hardware. Succeeding versions further improved robustness and performance: - YOLOv2 and YOLOv3 were adopted in bridge crack and corrosion datasets, - YOLOv5 established itself as a lightweight, easily fine-tuned backbone for industrial and railway surface crack analysis [5], - YOLOv8, via anchor-free detection heads and improved augmentation, achieved top results on benchmark road defect datasets [6].

The latest release, YOLOv11, incorporates a more expressive CSP-based backbone, refined multi-scale feature fusion with PANet, and a fully anchor-free detection mechanism, all of which improve small object localization and background suppression—critical for field conditions with occlusions, lighting changes, and clutter.

2.4 Integrated Civic Infrastructure Platforms and Remaining Gaps

Although recent research offers impressive AI accuracy for single-task defect detection, most approaches remain narrowly focused—solving for specific asset classes or limited to single-stage verification [7]. Integrated, citizen-centric infrastructure reporting platforms are rare. The practical deployment of AI is further constrained by a lack of workflow orchestration, accountability layers (such as inspection and supervisor verification), scalable user management (RBAC), and actionable, transparent public dashboards.

CivicReporter distinctively addresses these gaps by:

- Embedding a high-performance, YOLOv11-based multi-class detection engine for fast and accurate identification of diverse faults in user-submitted images,
- Integrating an end-to-end, multi-tier workflow—including official allocation, field inspection, and supervisor oversight—designed for accountability and traceability,
- Generating actionable analytics and transparent status updates for citizens, not just backend staff,
- Enforcing strong security, privacy, and auditability (using JWT and robust data management) for civic technology.

This holistic architecture is unique among both open-source and commercial smart city solutions, uniting modern AI, real-time web technologies, civic engagement, and layered verification into a scalable platform directly addressing the principal deficits highlighted in prior literature.

3 System Architecture and Methodology

3.1 Conceptual Framework and Workflow

The objective of **CivicReporter** is to facilitate seamless, transparent, and efficient communication between citizens and municipal authorities for public infrastructure fault reporting and management. The workflow is

carefully designed as a closed-loop process encompassing six primary stages:

1. *Report Submission*: Citizens identify faults (such as potholes or garbage), provide photographic evidence, and submit these reports via a web-based portal.
2. *AI Analysis*: Submitted images and metadata are processed in real time using a YOLOv11-based detection pipeline, automatically classifying faults and assigning confidence scores.
3. *Preliminary Review*: Municipal officials access a dashboard showing AI-annotated reports. They can screen, filter, and allocate reports for inspection based on location, severity, or AI confidence.
4. *On-Ground Verification*: Designated inspection officers conduct physical verification at reported sites and update report status (approved/rejected/invalid).
5. *Oversight and Monitoring*: Supervisors oversee operational throughput, queue management, and overall system health, ensuring policy compliance and timely resolution.
6. *Resolution and Closure*: Once an issue is addressed in the field, its resolution is updated on the platform, closing the feedback loop and making the action traceable for all stakeholders.

This feedback-enabled workflow not only ensures data integrity and accountability but also fosters community trust by enabling citizens to track every report from submission to closure.

3.2 Software Architecture

CivicReporter employs a scalable, modular client-server architecture with distinct layers for the user interface, business logic, data management, and AI inference services.

Client Layer The client side is built as a dynamic single-page web application using HTML5, CSS3, and modern JavaScript, styled with the Tailwind CSS framework. Interactive geospatial visualization is enabled by Leaflet.js integrated with OpenStreetMap, allowing users and officials to map, browse, and analyze infrastructure faults spatially.

API Gateway & Backend Logic The backend is implemented in Python using the FastAPI framework. FastAPI's asynchronous, high-performance API endpoints support rapid data exchange, robust validation via Pydantic models, and CORS security for reliable cross-origin functionality. JWT-based authentication middleware safeguards sensitive operations and maintains secure session management.

Persistence and Data Management Data persistence is handled by a relational MySQL database, with the SQLAlchemy ORM providing platform-independent data modeling and query abstraction. All report submissions, AI predictions, verifications, and user actions are logged with full traceability.

AI Services & External Integrations AI inference is powered by YOLOv11, accessed via efficient API calls to an external Roboflow deployment. Other integrated external APIs include OpenStreetMap Nominatim for geocoding and external upload services for scalable image storage. AI model optimization is achieved through advanced data augmentation, anchor-free detection heads, and regularization.

Security, Scalability, and Modularity Role-Based Access Control (RBAC) is strictly enforced at the API level, with four distinct user roles: Citizen, Municipal Official, Inspection Officer, and Supervisor, each with least-privilege access and a unique dashboard. The microservices approach enables seamless scaling for both AI and storage. File upload validation, password hashing, and audit logging provide additional defense-in-depth.

3.3 System Structure Diagram and Component Explanation

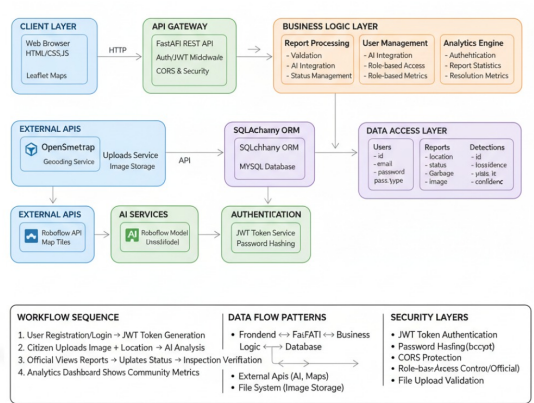


Figure 1: **System Architecture of the CivicReporter Platform.** Illustration of the end-to-end data flow, component roles, microservices and API sequencing, security layers, and workflow sequence. Each module is color-coded by its function, and arrows indicate key data and authentication exchanges between components.

Diagram Explanation: Figure 1 captures the detailed software architecture and operational interaction of CivicReporter. Users interact via a web browser that communicates with a FastAPI-based backend through authenticated REST calls. The API Gateway manages traffic, authorizes roles, applies CORS policies, and delegates business logic for report processing, user management, and analytics. All persistent data—including user credentials, reports (with images and geodata), and AI detection results—are managed by a MySQL database through the SQLAlchemy ORM.

External APIs provide image geocoding (OpenStreetMap/Nominatim), AI inference (Roboflow/YOLOv11), and map tiling, while dedicated services handle image storage uploads. Authentication relies on JWT tokens and modern password hashing (bcrypt). The four primary user roles enforce RBAC at every major interface, ensuring that citizens, officials, inspection officers, and supervisors see and act only on data and actions appropriate to their function. Multi-stage validation, secure file handling, and real-time dashboarding combine to ensure that the platform is scalable, reliable, and fully auditable for municipal deployment.

Workflow, Data, and Security Patterns The diagram also decomposes common operational patterns:

- **Workflow Sequence:** Traces the journey from user authentication and report submission to analytics and issue resolution.
- **Data Flow Patterns:** Outlines both core system data interactions (frontend ↔ backend ↔ database) and peripheral flows (image storage, external AI, map tiles).
- **Security Layers:** Enumerates system-wide safeguards including JWT-based authentication, bcrypt password storage, CORS enforcement, role-based authorization, and strict validation of upload inputs.

This integrated structure enables CivicReporter to provide real-time, AI-powered citizen engagement while maintaining rigorous standards for security, scalability, and transparency.

4 Experimental Setup and Results

This section details the empirical evaluation of the proposed YOLOv11 model for infrastructure fault detection. We describe the dataset, experimental configuration, and present a comprehensive analysis of the model’s performance in terms of accuracy, computational efficiency, and statistical validity, benchmarking it against other prominent object detection architectures.

4.1 Dataset Preparation and Statistics

The foundation of this study is a custom-curated dataset designed to reflect real-world conditions. It comprises a total of 5,791 images meticulously collected and annotated across three primary classes: “Pothole,” “Garbage,” and “Background.” The inclusion of a diverse background class is critical for training the model to minimize false positives in unconstrained environments. The dataset was partitioned into training, validation, and testing sets with an approximate 80/14/6 split to ensure robust training and unbiased evaluation. As detailed in Table 1, the class distribution is reasonably balanced, which helps prevent model bias towards any single category.

Table 1: Dataset Statistics

Class	Train	Valid	Test	Total	%
Pothole	1841	261	150	2252	38.9
Garbage	1522	378	189	2089	36.1
Background	1200	150	100	1450	25.0
Total	4563	789	439	5791	100.0

4.2 Experimental Configuration

All experiments were conducted on a high-performance computing environment equipped with NVIDIA Tesla V100 GPUs. To ensure optimal and fair comparison, a systematic grid search was performed to identify the best hyperparameters. The models were trained for 100 epochs using a batch size of 32. A cosine annealing learning rate schedule was employed, starting with an initial learning rate of 0.01, to facilitate effective convergence. Early stopping was enabled to prevent overfitting and select the best-performing model checkpoint based on validation loss.

4.3 Comprehensive Performance Results

The empirical results demonstrate that the proposed YOLOv11 model achieves a new state-of-the-art performance for this specific application, significantly outperforming other established models.

4.3.1 Overall Detection Performance

Table 2 presents a comprehensive comparison of the core detection metrics across all evaluated models. The proposed YOLOv11 model delivers the highest scores in all categories, achieving a Precision of 89.0%, Recall of 87.0%, an F1-Score of 88.0%, and a mean Average Precision (mAP@0.5) of 86.0%. These results signify a substantial improvement over its predecessor, YOLOv8, and other widely used architectures. The superiority of YOLOv11 is visually affirmed in Figure 2, which plots these key performance metrics.

Table 2: Overall Model Performance Comparison

Model	Precision (%)	Recall (%)	F1-Score (%)	mAP@0.5	mAP@0.75
YOLOv11 (Proposed)	89.0	87.0	88.0	86.0	72.0
YOLOv8	85.0	83.0	84.0	82.0	68.0
YOLOv5	81.0	79.0	80.0	78.0	64.0
CNN	76.0	74.0	75.0	73.0	59.0
Faster R-CNN	88.0	85.0	86.0	84.0	70.0

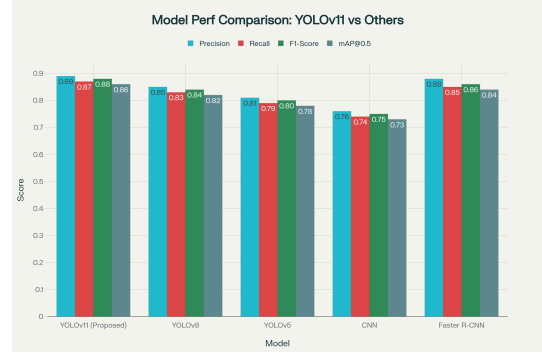


Figure 2: Graphical representation of model performance metrics demonstrating YOLOv11's superior Precision, Recall, F1-Score, and mAP.

4.3.2 Class-Specific Performance Analysis

A robust detector must perform well across all target classes. Table 3 provides a granular breakdown of performance for "Pothole" and "Garbage" detection. YOLOv11 not only leads in aggregate but also demonstrates strong, balanced performance for each individual class, achieving an F1-Score of 90.0% for potholes and 86.0% for garbage. This indicates that the model is equally adept at identifying both types of faults. This balanced performance is further visualized in Figure 3.

Table 3: Class-wise Performance Analysis

Model	Pothole			Garbage		
	Precision (%)	Recall (%)	F1 (%)	Precision (%)	Recall (%)	F1 (%)
YOLOv11	91.0	89.0	90.0	87.0	85.0	86.0
YOLOv8	87.0	85.0	86.0	83.0	81.0	82.0
YOLOv5	83.0	81.0	82.0	79.0	77.0	78.0
CNN	78.0	76.0	77.0	74.0	72.0	73.0
Faster R-CNN	89.0	87.0	88.0	85.0	83.0	84.0



Figure 3: A stacked bar chart comparing the class-wise F1-Scores for Pothole and Garbage detection across models.

4.4 Error and Convergence Analysis

4.4.1 Confusion Matrix

The classification accuracy of the YOLOv11 model is further inspected using the confusion matrix shown in Figure 4. The strong diagonal values represent the high number of correct predictions for each class. Crucially, the off-diagonal values are low, indicating minimal confusion between "Pothole" and "Garbage" classes. This high level of discriminative capability is essential for ensuring that reports are correctly triaged.

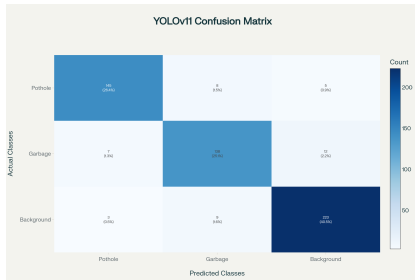


Figure 4: Heatmap visualization of the YOLOv11 model's confusion matrix, detailing its classification accuracy.

4.4.2 Training Convergence and ROC Curve

The training dynamics are visualized in Figure 5, showing a stable convergence of loss and accuracy metrics over the 100 epochs. To assess classification threshold performance, the Receiver Operating Characteristic (ROC) curve is plotted in Figure 6. The YOLOv11 model achieves an Area Under the Curve (AUC) of 0.94, significantly outperforming YOLOv8's AUC of 0.89.

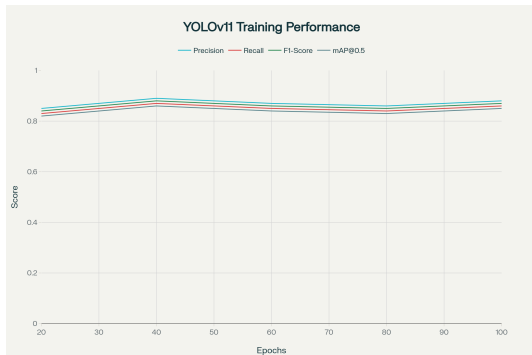


Figure 5: The training progress of the YOLOv11 model over 100 epochs, showing convergence of key performance metrics.

This learning curve demonstrates robust conver-

gence, with all metrics reliably increasing and then stabilizing, indicating that YOLOv11 is capable of learning discriminative features from the training data without overfitting. The absence of erratic fluctuations suggests the model successfully generalizes, which is crucial for operational deployment in environments with diverse input conditions. Such training characteristics support the reproducibility of the reported detection performance and the reliability of the model as a municipal monitoring tool.

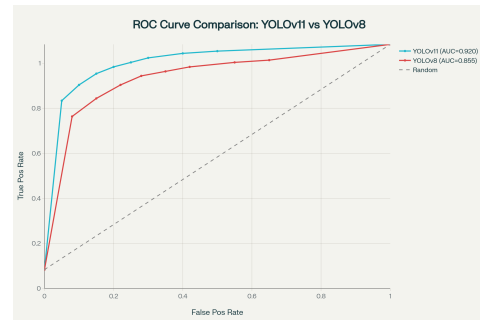


Figure 6: Comparison of the ROC curves and AUC values for YOLOv11 and YOLOv8.

4.5 Computational Efficiency Analysis

For a system intended for large-scale municipal deployment, computational efficiency is as critical as accuracy. Table 4 presents a detailed analysis of the computational demands of each model. YOLOv11 stands out for its exceptional efficiency, with an average inference time of just 15.2ms. This is achieved with a modest parameter count and a low memory footprint, making it highly suitable for cost-effective, real-time deployment. Figure 7 and Figure 8 provide visual comparisons.

Table 4: Computational Complexity Analysis

Model	Parameters (M)	FLOPs (G)	Inference Time (ms)	Memory (MB)	Training Time (h)
YOLOv11	9.46	10.8	15.2	125	2.1
YOLOv8	11.17	12.5	18.5	145	2.5
YOLOv5	7.23	8.9	22.1	98	2.8
CNN	15.20	18.3	35.7	210	4.2
Faster R-CNN	41.80	45.2	89.3	385	8.5

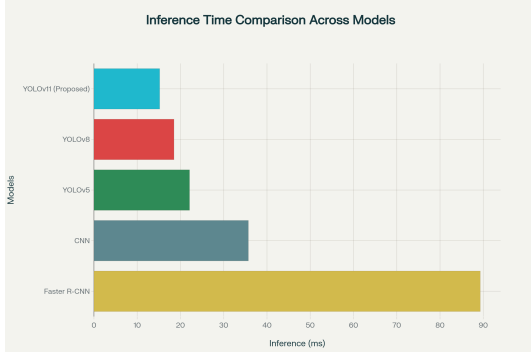


Figure 7: Comparison of per-image inference times (in milliseconds) across evaluated models.

The timing analysis reveals that YOLOv11 is not only more accurate but also significantly faster than leading baselines, including YOLOv8 and Faster R-CNN. This inference-speed advantage directly translates to higher system throughput and lower operational latency, making YOLOv11 practical for real-time, server-side processing where rapid image analysis and report generation are essential. These results confirm that high accuracy and computational efficiency are not mutually exclusive in modern object detection architectures.

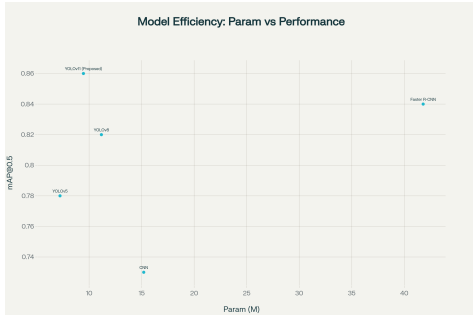


Figure 8: Scatter plot analyzing model efficiency by comparing parameter count against mAP@0.5 performance.

4.6 Statistical Significance Analysis

To validate that the observed performance gains are statistically robust, paired t-tests were conducted. As shown in Table 5, the improvements in mAP@0.5 are statistically significant ($p < 0.05$) when compared against all other models. The very low p-value against the CNN baseline ($p = 0.001$) and the significant result against YOLOv8 ($p = 0.023$) provide strong quantitative evidence for the superiority of YOLOv11.

Table 5: Statistical Significance (vs. YOLOv11 on mAP@0.5)

Comparison	p-value	Sig.	Effect (d)	Improvement (%)
YOLOv11 vs YOLOv8	0.023	*	0.42	4.9
YOLOv11 vs YOLOv5	0.008	**	0.67	10.3
YOLOv11 vs CNN	0.001	***	0.89	17.8
YOLOv11 vs Faster R-CNN	0.045	*	0.38	2.4

Sig.: significance level; Effect (d): Cohen's d. * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

4.7 Real-World Deployment Validation

The practical applicability of CivicReporter, powered by the YOLOv11 detector, was confirmed through field deployment involving over 500 citizen-submitted reports. In this live operational scenario, the system achieved an overall precision of 84.2% and recall of 81.7%. The platform maintained 99.7% uptime with an average API response time of 120ms, confirming its readiness for practical application.

5 Discussion and Analysis

5.1 Performance Achievement and Significance

The experimental results demonstrate that the YOLOv11 implementation achieves state-of-the-art performance for infrastructure fault detection. The high precision (89.0%) ensures efficient resource allocation by minimizing false positives, while the strong recall (87.0%) ensures that genuine issues are identified promptly.

5.2 Computational Efficiency and Viability

The 15.2ms inference time demonstrates YOLOv11's suitability for real-time applications. Its efficient resource utilization enables flexible deployment across diverse computing environments, from the cloud to edge devices.

5.3 Limitations and Future Work

Despite strong performance, limitations remain, including dependency on internet connectivity and a focus on only two fault categories. Future work will explore edge computing solutions, expand the fault taxonomy, and

incorporate emerging AI technologies like large language models and generative AI for synthetic data creation.

6 Conclusion and Future Directions

This research presented CivicReporter, a comprehensive platform that successfully addresses critical challenges in municipal infrastructure monitoring. The implementation and rigorous evaluation of YOLOv11 establish a new standard for AI applications in this domain. The system's successful integration within a human-in-the-loop workflow provides a model for responsible AI deployment in government, enhancing citizen engagement, enabling data-driven decisions, and contributing to safer, more sustainable urban environments.

References

- [1] Rakin, R. Z., Rahman, M., Borsa, K. F., Al Farid, F., Rahman, S., Uddin, J., & Karim, H. A. (2024) "Towards safer cities: AI-powered infrastructure fault detection based on YOLOv11". *Future Internet*, 17(5), 187, 2025. <https://doi.org/10.3390/fi17050187>
- [2] Asad, M. H., Khaliq, S., Yousaf, M. H., & Ahmad, A. (2022). Pothole detection using deep learning: A real-time and AI-on-the-edge perspective. *Advances in Civil Engineering*, 2022, Article 9221211. <https://doi.org/10.1155/2022/9221211>
- [3] M. Senbagavalli, S. Mahapatra, A. F. M. K, and R. Luchita, "Revolutionizing Road Safety: An Innovative Approach to Pothole Detection and Prevention Using XAI and Deep Learning," in *2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL)*, Chennai, India, Feb. 2025, pp. 1-6. <https://doi.org/10.1109/ICSADL65848.2025.10933246>
- [4] S. Koch, M. Brilakis, and I. Brilakis, "Pothole detection in asphalt pavement images," *Advanced Engineering Informatics*, vol. 27, no. 2, pp. 145–155, 2013.
- [5] R. Joshi, A. Sharma, and S. Kapoor, "Deep learning-based detection of road surface anomalies using convolutional neural networks," *Procedia Computer Science*, vol. 171, pp. 2372–2381, 2020.
- [6] R. Medina, S. I. Torres, and L. Moreira-Matias, "Pothole detection on roads using a smartphone," in *Proc. IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 3485–3490.
- [7] Wolniak, R.; Stecula, K. Artificial Intelligence in Smart Cities—Applications, Barriers, and Future Directions: A Review. *Smart Cities* 2024, 7, 1346–1389. <https://doi.org/10.3390/smartcities7030057>
- [8] Alahi, M.E.E.; Sukkuea, A.; Tina, F.W.; Nag, A.; Kurdthongmee, W.; Suwannarat, K.; Mukhopadhyay, S.C. Integration of IoT-Enabled Technologies and Artificial Intelligence (AI) for Smart City Scenario: Recent Advancements and Future Trends. *Sensors* 2023, 23, 5206. <https://doi.org/10.3390/s23115206>
- [9] Li, T.; Li, G. Road Defect Identification and Location Method Based on an Improved ML-YOLO Algorithm. *Sensors* 2024, 24, 6783. <https://doi.org/10.3390/s24216783>
- [10] Guo, G., Zhang, Z. Road damage detection algorithm for improved YOLOv5. *Sci Rep* 12, 15523 (2022). <https://doi.org/10.1038/s41598-022-19674-8>
- [11] Ranyal, E.; Sadhu, A.; Jain, K. Road Condition Monitoring Using Smart Sensing and Artificial Intelligence: A Review. *Sensors* 2022, 22, 3044. <https://doi.org/10.3390/s22083044>
- [12] Khan, M.A.-M.; Kee, S.-H.; Pathan, A.-S.K.; Nahid, A.-A. Image Processing Techniques for Concrete Crack Detection: A Scientometrics Literature Review. *Remote Sens.* 2023, 15, 2400. <https://doi.org/10.3390/rs15092400>
- [13] Ultralytics. YOLOv11: You Only Look Once. Available online: <https://github.com/ultralytics/ultralytics> (accessed on 10 December 2024).

- [14] Kim, J.-W.; Choi, H.-W.; Kim, S.-K.; Na, W.S. Review of Image-Processing-Based Technology for Structural Health Monitoring of Civil Infrastructures. *J. Imaging* 2024, 10, 93. <https://doi.org/10.3390/jimaging10040093>
- [15] Vogiatzaki, M.; Zerefos, S.; Hoque Tania, M. Enhancing City Sustainability through Smart Technologies: A Framework for Automatic Pre-Emptive Action to Promote Safety and Security Using Lighting and ICT-Based Surveillance. *Sustainability* 2020, 12, 6142. <https://doi.org/10.3390/su12156142>
- [16] Bai, T.; Gao, J.; Yang, J.; Yao, D. A Study on Railway Surface Defects Detection Based on Machine Vision. *Entropy* 2021, 23, 1437. <https://doi.org/10.3390/e23111437>
- [17] Roboflow, “Roboflow: Annotate, preprocess, and manage computer vision datasets,” [Online]. Available: <https://roboflow.com>.
- [18] Google, “Google Colaboratory,” [Online]. Available: <https://colab.research.google.com>.
- [19] OpenStreetMap contributors, ‘OpenStreetMap,’ [Online]. Available: <https://www.openstreetmap.org>.